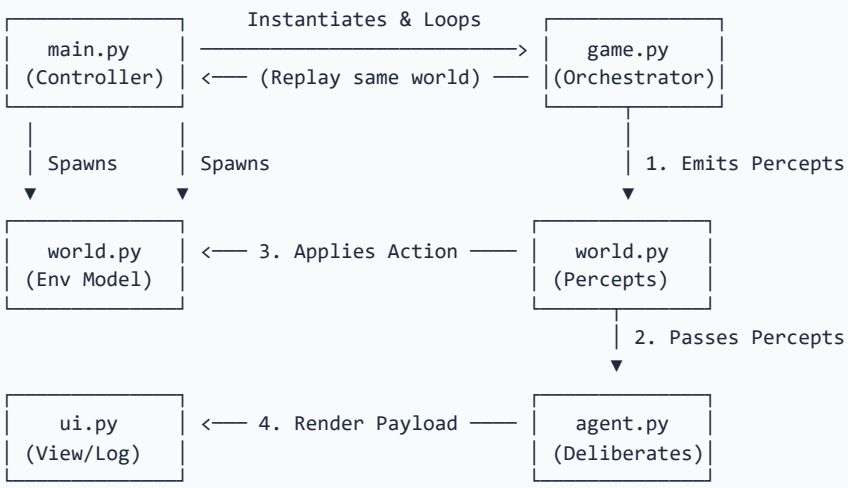
Wumpus World — Code Architecture & Technical Catalog

Complete Structural, Object-Oriented, and Behavioral Analysis | Autonomous Agent Simulation

> **Executive Summary:** The codebase implements a modular, autonomous Wumpus World AI simulation in Python. It enforces strict decoupling between environment simulation (`world.py`), artificial intelligence reasoning (`agent.py`), execution control (`game.py`), and presentation/logging (`ui.py`). Both Dumb (loop-safe random) and Smart (Knowledge Base + A* search) agents operate polymorphically through an abstract interface.

1. High-Level Architectural Flow & Patterns



Core Design Patterns Used

- **Model-View Separation:** Core logic modules (`World` , `Agent` , `Game`) have *zero terminal or string-drawing dependencies*. They return structured data dictionaries consumed by `ui.py` .
- **Strategy Pattern:** `Agent` is an Abstract Base Class. `DumbAgent` and `SmartAgent` implement identical `decide(percepts, world)` signatures, allowing seamless runtime interchangeability.
- **Deep State Cloning:** `World.clone()` provides deterministic replayability by creating deep independent copies of initial environments for direct agent benchmarking.

2. Comprehensive File, Class & Method Catalog

File: `world.py` Core Environment

Handles the grid coordinate system, hazard placement (Wumpus, Pits, Gold), sensory percept generation, arrow trajectory ray-casting, and deep cloning.

Symbol / Member	Belongs To	Behavior & Responsibility	Callers / Interactions
`turn_left(dir)` `turn_right(dir)`	Module `world.py`	Pure math rotation helpers: `(dir ± 1) % 4`. Numbers: 0=EAST, 1=NORTH, 2=WEST, 3=SOUTH.	`Agent.do_turn_left()`, `Agent.do_turn_right()`, `SmartAgent._turns_to_face()`
`World.__init__(rows, cols, seed)`	Class `World`	Initializes board. Generates isolated `random.Random(seed)`, places Wumpus, Gold, and Pits (scaled to grid size) on non-(0,0) tiles.	`main.main()`, `tests.py`
`World.is_valid(pos)`	Class `World`	Boundary check returning `True` if `0 <= r < rows` and `0 <= c < cols`.	`World.adjacent()`, `World.resolve_arrow()`, `Game._apply_action()`
`World.adjacent(pos)`	Class `World`	Returns list of valid orthogonal neighbor coordinates `(r+dr, c+dc)`.	`World.get_percepts()`
`World.get_percepts(pos, bump, scream)`	Class `World`	Calculates sensory dict: `stench` (living Wumpus neighbor), `breeze` (pit neighbor), `glitter` (gold on cell), `bump`, `scream`.	`Game.run()`, `tests.py`
`World.resolve_arrow(from_pos, dir)`	Class `World`	Ray-casts along facing vector. If hits living Wumpus, sets `is_wumpus_alive = False` and returns `True`. Otherwise returns `False`.	`Game._apply_action()`, `tests.py`
`World.clone()`	Class `World`	Executes `copy.deepcopy(self)` for reproducible replay sessions.	`main.main()`, `tests.py`

File: `agent.py` Agent AI & Knowledge Base

Contains the abstract `Agent`, loop-safe `DumbAgent`, `KnowledgeBase` inference engine, A* search, and `SmartAgent`.

Symbol / Member	Belongs To	Behavior & Responsibility	Callers / Interactions
`_astar(start, goal, walkable_fn, neighbors_fn)`	Module `agent.py`	Heuristic pathfinding with Manhattan distance: $h(n) = \|r_1 - r_2\| + \|c_1 - c_2\|$. Returns optimal tile path or `[]`.	`SmartAgent._plan_to_nearest()`, `SmartAgent.decide()`

Symbol / Member	Belongs To	Behavior & Responsibility	Callers / Interactions
`Agent.__init__()`	Class `Agent(ABC)`	Initializes pos `(0,0)`, direction `EAST`, arrow `True`, gold `False`, alive `True`, visited `{(0,0)}`.	Subclass constructors
`Agent.do_turn_left()` `Agent.do_turn_right()`	Class `Agent(ABC)`	Mutates `self.direction` using `turn_left()` / `turn_right()`.	`Game._apply_action()`
`Agent.get_forward_pos()`	Class `Agent(ABC)`	Computes target coordinate 1 step ahead in current direction.	`Game._apply_action()`, `DumbAgent.decide()`
`DumbAgent.decide(percepts, world)`	Class `DumbAgent`	Grabs gold on glitter; prefers unvisited neighbors; falls back to random turn/move when surrounded by visited tiles to escape loops.	`Game.run()`
`KnowledgeBase.mark_visited(pos, percepts)`	Class `KnowledgeBase`	Records percepts at tile, clears Wumpus threat if scream observed, and triggers `_deduce()`.	`SmartAgent.decide()`
`KnowledgeBase._deduce()`	Class `KnowledgeBase`	Propagates deductive rules: zero hazards mark neighbors `SAFE`; single unexplained stench marks `CONFIRMED_WUMPUS`; breeze marks `CONFIRMED_PIT`.	Internal via `mark_visited()`
`KnowledgeBase.is_safe(pos)`	Class `KnowledgeBase`	Returns `True` if tile is marked `SAFE` or `VISITED`.	Passed to `_astar` walkable callback
`SmartAgent.decide(percepts, world)`	Class `SmartAgent`	1. Updates KB; 2. Grabs gold if glitter; 3. Pops planned action; 4. Shoots Wumpus if aligned; 5. A* plans to nearest safe unvisited tile; 6. Evaluates minimum risk.	`Game.run()`

Symbol / Member	Belongs To	Behavior & Responsibility	Callers / Interactions
`SmartAgent._plan_to_nearest(targets)`	Class `SmartAgent`	Finds nearest target via A*, converts tile path to turn/move actions, and formats human reasoning string.	`SmartAgent.decide()`

File: `game.py` Engine & Rules

Executes turns, applies movement/shooting physics, tallies scores, and detects victory/death conditions.

Symbol / Member	Belongs To	Behavior & Responsibility	Callers / Interactions
`Game.__init__(world, agent, max_steps)`	Class `Game`	Initializes session state, resets score to 0, step counter to 0, and status to `None`.	`main.main()`, `tests.py`
`Game.run()`	Class `Game`	Primary game loop: gathers percepts, requests agent decision, applies action (-1 pt), renders step to UI, evaluates end state, caps at `max_steps`.	`main.main()`, `tests.py`
`Game._apply_action(action)`	Class `Game`	Handles `MOVE_FORWARD` (validates & moves or triggers bump), `TURN_LEFT`, `TURN_RIGHT`, `GRAB` (acquires gold), `SHOOT` (calls `world.resolve_arrow`, awards +1000 on kill or -10 on miss).	`Game.run()`
`Game._check_end_conditions()`	Class `Game`	Evaluates: live Wumpus contact (-1000, LOSE), Pit contact (-1000, LOSE), Gold possession (+1000, WIN), or Wumpus scream heard (+1000, WIN).	`Game.run()`

File: `ui.py` Presentation & Dual Logging

Renders ASCII/Unicode boards and mirrors all telemetry in real time to `result.txt`.

Symbol / Member	Belongs To	Behavior & Responsibility	Callers / Interactions
`open_result_file()` `close_result_file()`	Module `ui.py`	Opens `result.txt` for overwriting each session; ensures clean buffer flush and close upon exit.	`main.main()`
`_print(*args, **kwargs)`	Module `ui.py`	Drop-in replacement for `print()` that simultaneously writes to `stdout` and `result.txt`.	All rendering functions in `ui.py` and `main.py`
`render_grid(...)`	Module `ui.py`	Draws Unicode box grid (`┌┬┐`). Displays agent arrows (`► ▲ ◄ ▼`), hazards (`W, P, G`), and hides unknown cells as `?` unless `reveal_all=True`.	`Game.run()`, `render_game_over()`
`render_step_log(...)`	Module `ui.py`	Outputs labeled telemetry blocks: `[PERCEPT]`, `[THINK]`, `[ACTION]`, `[SCORE]`.	`Game.run()`
`render_game_over(...)`	Module `ui.py`	Prints game summary outcome, metrics, and renders full board with `reveal_all=True`.	`Game.run()`
`prompt_grid_size()` `prompt_agent_mode()` `prompt_post_game()`	Module `ui.py`	Interactive CLI menus handling input parsing with safe default fallbacks.	`main.main()`

File: `main.py` Application Entry Point

Symbol / Member	Belongs To	Behavior & Responsibility	Callers / Interactions
`create_agent(mode, rows, cols)`	Module `main.py`	Factory function: creates `SmartAgent` when mode=2, else `DumbAgent`.	`main.main()`
`main()`	Module `main.py`	Session manager: manages replay loop (same world via `World.clone()`, new world, or exit).	Script execution entry point

3. Cross-Module Invocation & Dependency Graph

Caller Method / Function	Source File	Target Object / Method	Target File	Purpose & Effect
`main()`	`main.py`	`World.clone()`	`world.py`	Produces deep copy of generated world for identical re-runs.
`Game.run()`	`game.py`	`World.get_percepts(...)`	`world.py`	Fetches sensory flags (stench, breeze, glitter, bump, scream).
`Game.run()`	`game.py`	`Agent.decide(percepts, world)`	`agent.py`	Invokes strategy method returning `(action, thought)`.
`Game._apply_action()`	`game.py`	`Agent.get_forward_pos()`	`agent.py`	Calculates destination cell before applying kinematic step.
`Game._apply_action()`	`game.py`	`World.is_valid(pos)`	`world.py`	Validates if forward movement collides with boundary wall.
`Game._apply_action()`	`game.py`	`World.resolve_arrow(...)`	`world.py`	Simulates arrow flight; updates `is_wumpus_alive` if hit.
`SmartAgent.decide()`	`agent.py`	`KnowledgeBase.mark_visited()`	`agent.py`	Ingests sensory data and executes deduction propagation.
`SmartAgent._plan_to_nearest()`	`agent.py`	`_astar(...)`	`agent.py`	Calculates shortest safe path to unexplored frontier.